데이터셋 선정과 R0 검색의 효과: QASPER에서 Qwen2.5와 Qwen3.8 비교

작성일: 2026-11-08 · 기존 실험 원자료를 확인하고 재집계한 보고서

이 프로젝트는 **LLM에게 자료 전체를 보내는 대신, 질문에 필요한 부분을 찾아 보내면 입력량을 얼마나 줄이고 답변 품질을 얼마나 유지할 수 있는가**를 연구한다. LLM은 입력된 글을 읽고 답변을 만드는 대규모 언어 모델이며, **R0는 그 모델에 보낼 원문을 고르는 검색 방식**이다. 검색 방식과 답변 모델은 서로 바꿀 수 있는 별개의 구성 요소다.

이번 보고서는 ① 검토한 데이터셋 7종, ② QASPER 선정 이유, ③ 기존 Qwen2.5에서의 R0 결과, ④ Qwen3.8로 교체한 결과를 순서대로 설명한다. 데이터셋은 평가용 질문·자료·참조 답안을 모은 묶음이다. 이번에는 이를 **모델을 새로 학습시키는 용도가 아니라, 동일한 문제로 성능을 비교하는 용도**로 사용했다.

핵심 관측은 두 가지다. Qwen2.5에서 R0는 논문 전체 입력보다 입력 토큰을 **77.2%** 줄였지만 F1은 **10.38점** 낮았다. 같은 R0 원문을 Qwen3.8에 전달하자 F1은 **34.13 → 36.45점**으로 올랐고, 평균 답변 호출 시간은 **3.69 → 12.69초**로 늘었다. F1과 시간의 정확한 의미는 3·4절에서 구분한다.

1. 검토한 데이터셋 7가지

1.1 각 데이터셋은 무엇을 평가하는가?

'긴 자료를 잘 읽는다'는 능력에도 차이가 있다. 한 문서에서 사실을 찾는 일, 여러 문서의 단서를 연결하는 일, 회의록을 요약하는 일은 서로 다른 과제다. 그래서 처음에는 다음 7종을 검토했다.

데이터셋	입력 자료와 요구하는 답변	주로 확인하는 능력	준비한 고유 평가 문항
QMSum	회의록에서 질문과 관련된 발언을 찾아 요약	긴 대화에서 관련 내용 선택·종합	272
QASPER	자연어처리 연구 논문을 읽고 질문에 답함	기술 문서의 근거 검색·설명 이해	984
NarrativeQA	책·영화 대본의 원문을 읽고 질문에 답함	긴 이야기의 사건·인물·관계 이해	3,425
QuALITY	긴 글을 읽고 네 선택지 중 정답을 고름	글 전반의 이해와 선택지 판단	2,086
HotpotQA	여러 문서의 정보를 결합해 질문에 답함	문서 사이의 단서 연결	200
MuSiQue	앞 단계의 정보를 이용해 다음 단서를 찾는 질문에 답함	여러 단계가 연결된 추론	200
MultiFieldQA-en	다양한 분야의 긴 영어 문서에서 답을 찾음	분야가 달라져도 필요한 근거를 찾는 능력	150

위 수는 각 데이터셋의 전체 공개 규모가 아니라 **이 프로젝트가 준비한 판본의 문항 수**다. 앞의 네 종류는 SCROLLS의 공개 검증 세트, 뒤의 세 종류는 LongBench v1의 평가 세트다. LongBench의 200문항을 원본 HotpotQA·MuSiQue 전체라고 해석하면 안 된다. SCROLLS와 LongBench는 여러 과제를 일정한 입력·평가 형식으로 모은 평가 모음이다. 자료별 판본·문항 수, SCROLLS 논문, LongBench 공식 설명

1.2 코딩 연구와 어떤 관계가 있는가?

이 7종은 **코드 작성·수정·실행을 직접 채점하는 데이터셋이 아니다.** QASPER는 기술적인 문서를 이해한다는 점에서 코딩 관련 설명을 읽는 작업과 가깝고, MuSiQue·HotpotQA는 흩어진 정보를 연결한다는 점에서 여러 파일을 따라가는 작업과 닮았다. 다만 문서의 단서 연결이 실제 함수 호출이나 `import` 관계를 대신하지는 않는다. QASPER 논문, MuSiQue 논문

또한 **RAPTOR 논문이 사용한 것은 이 중 QASPER·NarrativeQA·QuALITY의 3종**이다. 7종 모두를 'RAPTOR 데이터셋'이라고 부르면 부정확하다. RAPTOR는 여러 원문을 묶고 반복해서 요약해 계층을 만든 뒤 검색하는 연구다. 이번 보고서는 그 연구가 사용한 데이터셋을 활용했으며, RAPTOR의 요약·군집 알고리즘을 실행한 결과는 아니다. RAPTOR 논문

실험 범위: 처음 검토한 대상은 7종이지만, 최종 완료 비교는 QASPER 하나로 좁혔다. 중단된 7종 실행의 일부 결과를 이번 수치에 합치지 않았다.

2. QASPER를 선택한 이유

2.1 원문 전체와 검색 결과를 공정하게 비교할 수 있다

검색으로 입력을 줄인 효과를 알려면 '원문 전체를 읽힌 경우'도 실제로 실행해야 한다. 전체 원문이 모델의 한도를 넘으면, 검색 방식과 원문 전체 방식이 서로 다른 질문을 처리하게 되어 비교가 복잡해진다.

토큰은 모델이 글을 나누어 처리하는 단위로, 단어나 글자 수와 정확히 같지 않다. **컨텍스트 윈도우**는 한 번의 호출에서 입력과 생성할 답변을 담을 수 있는 한도다. 이번 Qwen2.5 실험은 이를 32,768토큰으로 설정했다.

RAPTOR에서 사용한 세 후보의 공개 검증 자료를 같은 기준으로 점검한 결과는 다음과 같다.

후보	고유 질문	고유 원문	원문 전체를 입력할 수 있는 질문
QASPER	**984**	**273**	**984개, 100%**
QuALITY	2,086	118	2,086개, 100%
NarrativeQA	3,425	115	565개, 약 16.5%

QASPER는 지시문·질문·논문·답변 예약 128토큰을 모두 합쳐도 최대 **22,192토큰**이었다. 따라서 전체 984문항에서 검색 방식과 원문 전체 방식을 같은 분모로 비교할 수 있었다. QuALITY도 모두 입력할 수 있지만 문항 수가 더 많았고, NarrativeQA는 대부분 전체 원문이 한도를 넘었다. 이 판단은 **답변 점수를 보기 전의 자료 구조·규모·입력 길이**를 근거로 했다. 선정 근거, 문항별 입력 가능성 검사

2.2 구조가 있는 기술 문서에서 검색을 시험할 수 있다

QASPER는 논문 안의 방법·설정·결과 등을 찾아 답하는 과제다. 절 제목과 문단 위치가 있어, 자료를 조각으로 나누고 질문과 관련된 부분을 찾는 실험에 적합하다. 이번 판본의 원자료 1,726행에는 같은 질문에 대한 여러 참조 답안이 포함돼 있었다. 이를 질문 ID로 묶어 **984문항과 273개 고유 논문 원문**을 모두 사용했다. 자료 구성 기록

다만 질문마다 한 논문이 제공되므로 **파일 간 참조 탐색의 효과를 검증하기에는 한계가 있다.** 이번 문서용 B는 같은 논문 안에서 A보다 한 단계 더 연결을 따라간다. 따라서 QASPER 결과는 기술 문서 검색의 근거이며, 실제 코드 저장소에서의 성능을 그대로 입증하지 않는다.

3. Qwen2.5를 사용한 기존 구조와 R0의 QASPER 실험

3.1 R0는 누가, 무엇을 검색하는 방식인가?

R0에서 **우리 프로그램이 자료를 정리하고 검색하며, Qwen 모델은 골라진 원문을 읽고 답한다.** 모델이 검색 가중치를 받아서 스스로 원문 파일을 여는 구조는 아니다.

```
사전 준비 — 질문과 정답을 보지 않고 수행
논문 원문 → 최대 256토큰의 조각 → 조각 사이의 연결 정보
          → 단독 조각·연결 묶음을 검색 후보로 등록 → TF-IDF 계산

질문이 들어온 뒤
질문의 단어 분석 → 후보와 유사도 비교 → 예산 안에서 원문 선택
                 → 질문·지시문·선택한 실제 원문을 Qwen에 전달
                 → 답변 생성 → 참조 답안과 비교해 채점
```

여기서 **후보란 'LLM에 보낼지 판단하기 위해 검색 목록에 등록한 텍스트 항목 하나'**다. 한 원문 조각이 후보일 수도 있고, 출발 조각에서 연결을 최대 3단계 따라간 여러 조각의 묶음이 후보일 수도 있다. '3단계'는 조각을 반드시 3개만 넣는다는 뜻이 아니다. 가지가 갈라지면 더 많은 조각이 묶일 수 있다.

예를 들어 단독 후보 `조각 2`가 선택되면 그 원문만, 묶음 후보 `(조각 1·2·3)`이 선택되면 묶음의 원문을 보낸다. 실제 연결은 같은 긴 문단을 나눈 조각 사이의 이어짐과, 같은 논문 안에서 절 제목을 명시적으로 언급한 관계다. 다른 논문으로 연결되는 관계는 0개였다.

TF-IDF는 단어 빈도로 텍스트를 수치화하는 기존 정보검색 방법이다. TF는 후보 안에서 단어가 나타난 횟수, IDF는 전체 후보 중 그 단어가 얼마나 드물게 등장하는지를 반영한다. 이번 구현은 다음 가중치를 사용한다.

```
TF = 1 + ln(그 후보에서의 등장 횟수)        — 등장한 단어에 적용
IDF = ln((후보 수 + 1) / (그 단어가 있는 후보 수 + 1)) + 1
단어 가중치 = TF × IDF
```

`ln`은 자연로그다. 예를 들어 후보 10개 중 2개에 있는 단어의 IDF는 `ln(11/3)+1 ≈ 2.30`이고, 10개 모두에 있는 단어는 1이다. 드문 단어가 후보를 구별하는 데 더 큰 비중을 얻는 셈이다. 후보의 단어 가중치들을 한 벡터로 묶어 길이를 1로 맞추고, 질문도 **동일한 IDF**로 벡터를 만든다. 두 벡터에서 같은 단어의 값을 곱해 합하는 코사인 유사도로 순위를 정한다. 따라서 문맥을 완전히 이해해서 고르는 방식이 아니라 **단어의 일치와 분포로 관련성을 추정하는 검색**이다.

IDF는 해당 논문의 후보들로 계산하며, 후보에는 원문과 출처 ID·위치 표시가 포함된다. 높은 순위부터 **출처 표시를 포함한 1,024토큰 예산** 안에 넣는다. 후보 전체가 들어가지 않으면 건너뛰고, 이미 포함한 내용과 완전히 중복된 후보는 제외하지만 부분 중복은 남을 수 있다. 이 준비·검색 과정에는 LLM 요약 호출이 없다. 실제 검색 구현, 계산 예시를 포함한 상세 보고서

3.2 무엇과 비교했고, 조건은 어떻게 맞췄나?

방식	자료를 고르는 과정	질문당 모델 호출
A: 문서용 색인	프로그램이 위치·절 제목·앞 12단어 미리보기 목록을 추림 → LLM이 출발 조각 선택 → 같은 문서 연결 1단계 확장	선택 1회 + 답변 1회
B: 연결 확장	A와 같은 목록과 별도 선택 호출 → 연결 2단계 확장	선택 1회 + 답변 1회
R0: 원문 검색	프로그램이 TF-IDF로 단독·묶음 원문 후보를 고름	답변 1회

방식	자료를 고르는 과정	질문당 모델 호출
FULL: 원문 전체	검색하지 않고 해당 논문의 제공 원문을 전부 전달	답변 1회

A·B는 코드의 함수 이름·위치 색인을 **문서용으로 바꾼 방식**이다. 미리보기에는 원문의 앞부분이 포함되며, 실제 함수·`import`를 분석한 코드 실험과 구분한다. 답변 모델에는 A·B도 선택된 실제 원문을 보낸다.

공통 조건	설정
답변 모델·실행 엔진	**Qwen2.5-Coder-7B-Instruct Q5_K_M**, 로컬 llama.cpp 서버
평가 범위	동일한 984문항, 방식별 1회 생성, 총 3,936개 결과
컨텍스트·답변 한도	호출당 32,768토큰, 답변 최대 128토큰
검색 자료 예산	A·B·R0 모두 최대 1,024토큰; FULL은 전체 원문
A·B의 선택 목록	최대 24개·2,048토큰, 선택 출력 32토큰, 확장 최대 8조각
생성 조건·순서	`temperature=0`, `seed=1`, 프롬프트 캐시 비활성화; 네 방식의 24가지 실행 순서를 각각 41번 배치

`temperature=0`은 출력의 무작위성을 줄이는 설정이고, seed는 난수의 시작값이다. 프롬프트 캐시는 앞서 처리한 입력 계산을 재사용하는 기능으로, 여기서는 껐다. **1,024토큰은 자료만의 예산**이며, 지시문과 질문까지 포함한 실제 입력은 이보다 크다. 고정 실험 설정

3.3 점수·토큰·시간을 읽는 방법

여기서 F1으로 확인하려는 품질은 질문에 필요한 정보를 얼마나 빠짐없이 담으면서, 정답에 없는 내용을 얼마나 덜 섞어 답하는가이다. 논문 질의응답에서는 필요한 정보의 일부만 맞히는 답변도 나온다. 예를 들어 사용한 데이터셋 두 개를 묻는데 하나만 답하면, 사용자는 나머지를 직접 찾아야 한다. 두 개를 모두 말하면서 사용하지 않은 데이터셋까지 덧붙이면, 사용자는 어느 것이 맞는지 다시 확인해야 한다. F1은 이런 누락과 과잉을 함께 반영해 답변이 참조 답안에 얼마나 가까운지 점수로 나타낸다.

이 실험의 자동 채점은 **참조 답안과 겹치는 단어**를 이용해 그 품질을 근사한다. **정밀도**는 답변에 쓴 단어 중 참조 답안과 겹치는 비율로, 정답에 없는 표현을 덜 섞었는지 본다. **재현율**은 참조 답안의 단어 중 답변에 포함된 비율로, 필요한 표현을 얼마나 담았는지 본다. F1이 높으려면 두 비율이 함께 높아야 한다.

다음은 이 뜻을 설명하기 위한 가상 예시다. “사용한 데이터셋은 무엇인가?”의 참조 답안을 `SST-2, IMDb`라고 하자. 두 이름이 필요한 정보에 해당한다.

모델 답변	F1 / 100	답변을 읽는 사람에게 어떤 차이가 있는가
`SST-2`	66.67	맞는 이름 하나를 알려 주지만, 필요한 다른 하나가 빠졌다.
`SST-2, IMDb`	100.00	필요한 두 이름을 모두 정확하게 알려 준다.
`SST-2, IMDb, MNLI`	80.00	필요한 이름은 모두 있지만, 정답에 없는 이름까지 섞여 있다.
`MNLI`	0.00	필요한 두 이름을 모두 제공하지 못했다.

계산할 때는 소문자화, 구두점·영어 관사 제거, 공백 정리를 거친다. SST-2만 답한 경우 정밀도는 1/1, 재현율은 1/2이고, F1은 2 × 겹친 단어 수 / (답변 단어 수 + 참조 단어 수) = 2/3이다. 한 문항에 참조 답안이 여러 개이면 가장 높은 F1을 쓰고, 984문항의 평균에 100을 곱해 보고한다. 공식 SCROLLS 계산 규칙을 적용했다. 실제 채점 구현, 분석 계획

평균 F1이 오르면, 이 단어 비교 기준에서 답변들이 참조 답안에 더 가까워졌다고 해석한다. 실제로 빠진 정보가 보완되거나 잘못 덧붙인 내용이 줄었다면 사용자가 답변을 활용하는 데 도움이 된다. 다만 이 채점기가 직접 확인하는 범위는 단어의 일치다. 같은 뜻을 다른 표현으로 쓴 답도 점수가 달라지고, 단어가 같아도 순서나 부정 표현에 따라 문장의 뜻은 달라질 수 있다. 따라서 점수 상승을 실제 정보 개선과 연결하려면 응답 내용도 함께 확인해야 한다. 4.3절에서 이번 실험의 응답으로 그 차이를 보여준다.

누적 입력 토큰은 질문 하나를 처리하면서 모든 호출에 보낸 입력의 합이다. **최대 호출 입력의 평균**은 질문별로 가장 큰 한 호출의 입력을 고른 뒤 평균한 값이다. 예를 들어 선택에 300토큰, 답변에 700토큰을 쓰면 누적 입력은 1,000이고 최대 호출 입력은 700이다. 전자는 총 입력 비용, 후자는 한 번에 차지하는 컨텍스트를 비교한다. 출력 토큰은 둘 다에 포함하지 않는다.

아래 **총 질의 시간**은 검색·선택·원문 구성·답변을 포함한다. 모델 시작과 사전 색인·TF-IDF 구성·자료 로딩은 제외한다. 자료 분할·목록·연결 생성과 저장에는 별도로 **50.07초**가 걸렸다.

3.4 Qwen2.5 실험 결과

방식	평균 F1 / 100	누적 입력 토큰 / 질문	최대 호출 입력의 평균	평균 총 질의 시간
A	26.12	1,408.9	925.1	4.74초
B	26.72	1,504.3	1,000.4	5.03초
R0	**34.13**	**1,154.6**	**1,154.6**	**3.82초**
FULL	44.51	5,067.4	5,067.4	16.55초

R0의 원문 전체 대비 입력 절감률은 100 × (1 − 1,154.625 / 5,067.3801) ≈ 77.2%다. 그러나 F1은 34.1303 − 44.5076 ≈ −10.38점이었다. **입력·시간을 줄이는 이득과 답변 품질 손실이 함께 관측된 것**이다. R0는 A·B보다 F1이 높고 누적 입력·시간도 적었지만, 한 호출의 입력을 작게 유지하는 데에는 A가 더 유리했다.

동일 질문에서의 F1 차이	관측 차이	95% 구간
R0 − A	+8.01점	+5.81 ~ +10.27
R0 − B	+7.41점	+5.10 ~ +9.78
R0 − FULL	−10.38점	−12.44 ~ −8.31
B − A	+0.60점	−0.09 ~ +1.33

같은 논문의 질문끼리는 관련될 수 있으므로, 질문 하나씩이 아니라 **273개 원문 묶음 단위**로 중복을 허용해 10,000번 다시 뽑아 차이를 계산했다. 이를 짝지은 군집 부트스트랩이라고 하며, 위 구간은 계산된 차이들의 가운데 95% 범위다. B−A는 구간에 0을 포함해 우위를 분명하게 주장하기 어렵다. 이 구간은 불확실성 설명용이며 여러 비교를 보정한 확증 검정은 아니다. 재표집 seed는 20261027이다.

모든 방식이 984개 답변을 완료했다. A·B의 목록 선택 실패 각 10건도 빈 원문으로 받은 답변을 채점해 포함했고, 입력 초과·입력 잘림·답변 출력 한도 도달은 없었다. 전체 3,936개 결과와 5,904회 평가 호출을 전수 검증했다. 최종 집계, 검증 기록

4. Qwen3.8로 모델을 교체한 후의 결과

4.1 무엇을 유지하고 무엇을 바꿨나?

교체 대상의 정확한 이름은 **Qwen3.8-27B MLX 5bit**다. 3.8은 모델 계열 이름이며, 'Qwen3의 8B 모델'을 뜻하지 않는다. 7B와 27B는 학습된 수치인 매개변수가 각각 약 70억·270억 개라는 규모를 나타낸다. Qwen2.5 공식 모델 설명, Qwen3.8 공식 모델 설명

이번 교체 실험은 **R0 검색을 다시 실행해 다른 원문을 고른 것이 아니라, 기존 R0가 선택한 같은 원문과 질문을 새 모델에 다시 전달한 실험**이다. 따라서 '같은 근거를 줬을 때 답변이 어떻게 달라지는가'를 비교한다.

항목	기존	교체 후
모델	Qwen2.5-Coder-7B-Instruct	Qwen3.8-27B
가중치 저장·실행 형태	GGUF Q5_K_M · llama.cpp	MLX 5bit · MLX-LM
질문·참조 답안·선택 원문	QASPER 984문항의 R0 자료	같은 문항·참조 답안·원문 재사용
사용자 지시문·시스템 메시지	기존 QASPER 답변 지시문	같은 내용 유지
대화 포장 형식	기존 모델의 형식	새 모델 형식과 추론 비활성화용 표시 적용
생성 조건	최대 128토큰, temperature 0, seed 1	동일; 중간 추론 생성 비활성화
실행·캐시	순차 실행, 프롬프트 캐시 비활성화	동일

양자화는 모델 가중치를 더 적은 비트로 저장해 메모리 부담을 줄이는 방법이다. Q5_K_M과 MLX 5bit는 모두 양자화본이지만 **같은 저장·계산 방식은 아니다.** 모델 세대뿐 아니라 규모, 실행 엔진, 토큰을 나누는 방식, 대화 포장도 바뀌었다. 따라서 결과는 **이 두 실행 구성의 비교**이며, 모든 변화가 '3.8이라는 세대' 하나에서 비롯됐다고 분리해 말할 수는 없다.

교체 실행 기록의 장비는 Apple M3 Pro·36GiB 메모리이며, `mlx 0.32.4`, `mlx-lm 0.32.2`를 사용했다. 배포본 ID는 `lmstudio-community/Qwen3.8-27B-MLX-5bit`, 고정 리비전은 `b9ad7b92aa2fd07b041f4b8b3aa459941f8abbf7`이다. 이번 문서 작성에서는 모델을 다시 돌리지 않고, 보관된 984개 응답을 기존 입력 기록과 대조하고 재채점했다. 교체 실험 압축 원자료, 이번 재집계·대조 기록

4.2 같은 R0 원문을 읽었을 때의 변화

지표	Qwen2.5 + R0	Qwen3.8 + 같은 R0 원문	변화
평가·답변 완료 문항	984 / 984	984 / 984	동일
평균 F1 / 100	**34.13**	**36.45**	**+2.32점**
평균 입력 토큰 / 질문	1,154.6	1,161.9	+7.3토큰
평균 답변 호출 시간	3.69초	12.69초	+9.01초, 약 3.44배
답변 출력 한도 도달	0건	0건	동일

이 결과에서 좋아진 것은 같은 검색 자료를 읽고 참조 답안에 가까운 답변을 만들어 내는 정도다. 984문항의 평균 F1이 34.13점에서 36.45점으로 올랐으므로, 필요한 표현을 담고 정답에 없는 표현을 줄이는 균형이 단어 비교 기준에서 평균 2.32점 개선됐다. 실제 응답에는 기존 모델이 빠뜨린 수량을 보완하거나, 잘못 덧붙인 언어 이름을 제거한 사례가 있다. 이런 개선은 사용자가 질문한 정보를 더 온전하고 정확하게 받는다는 의미가 있다. 구체적인 응답은 4.3절에 제시한다.

100점 척도에서 **+2.32점은 평균적인 일치 정도의 소폭 상승**이다. 기존 점수를 기준으로 환산한 상대 증가 약 6.81%도 같은 점수 변화를 다른 단위로 표현한 것이다. 완전히 맞힌 문항 수를 알려면 문항별 정오를 별도로 판정해야 한다. 또한 F1에는 표현상의 일치도 반영되므로, 실제로 누락 정보나 잘못된 정보가 얼마나 줄었는지는 응답을 확인한 범위에서 해석한다. 차이는 모두 반올림 전 값으로 계산했다.

두 모델에 전달한 원문 내용은 같다. 입력 토큰이 약간 달라진 것은 모델별 토큰 분할과 대화 포장 형식이 다르기 때문이다. 기존의 1,024토큰 자료 예산으로 골라진 원문을 그대로 유지했으며, 새 모델의 토큰 계산으로 다시 자르거나 검색하지 않았다. 따라서 이 입력 차이를 검색 효율의 변화라고 해석하지 않는다.

시간은 사용자가 답변 생성을 기다리는 부담과 연결된다. 이번 측정에서 **평균 F1 2.32점의 상승과 함께, 답변 한 번을 생성하는 시간이 평균 9.01초 더 걸렸다.** 따라서 이 교체는 같은 자료에서 답변의 평균 일치 정도를 조금 높이는 대신 대기 시간이 약 3.44배 늘어난 결과다. 그 품질 이득이 대기 시간을 감수할 만큼 유용한지는 실제 응답 개선 사례와 함께 판단해야 한다.

시간의 측정 범위도 구분해야 한다. 3절의 R0 **3.82초는 검색을 포함한 총 질의 시간**, 이 절의 **3.69초와 12.69초는 각각 기존·새 모델의 답변 호출 시간**이다. 새 실험은 이미 골라 둔 원문으로 답변을 생성했다. 기존 모델을 같은 날 다시 실행한 동시 대조 실험도 아니므로, 시간에는 실행 엔진과 당시 시스템 상태의 영향이 함께 들어갈 수 있다.

4.3 평균 향상이 모든 질문의 개선을 뜻하는가?

질문마다 얻는 이득은 달랐다. 전체의 약 19.5%에서 답변이 참조 답안에 더 가까워졌고, 약 19.2%에서는 더 멀어졌다. 따라서 평균 상승만으로 개별 질문의 답변도 좋아질 것이라고 기대하기에는 편차가 크다.

문항별 F1 변화	문항 수	전체에서 차지하는 비율	해당 문항들의 평균 F1 변화
상승	192	19.5%	+40.86점
동일	603	61.3%	0.00점
하락	189	19.2%	−29.40점
합계	984	100.0%	+2.32점

상승한 문항과 하락한 문항의 수는 비슷하지만, **상승한 문항에서 얻은 점수가 하락한 문항에서 잃은 점수보다 컸다.** 상승군은 평균 40.86점 올랐고 하락군은 평균 29.40점 내려, 전체 평균에 +2.32점이 남았다. 동일한 603문항은 F1 변화가 0인 경우를 모은 것으로, 답변 내용의 동일성이나 정답 여부는 별도로 확인해야 한다.

실제 응답에서는 다음과 같은 변화가 있었다. 질문은 한국어로 요약했고, 답변은 기록에 있는 영어 표현을 그대로 옮겼다.

질문	Qwen2.5에서 Qwen3.8로 바뀐 응답과 F1	사용자에게 어떤 의미가 있는가
Twitter 데이터셋의 규모는 얼마인가?	`1,873 conversation threads` → `1,873 Twitter conversation threads, roughly 14k tweets` · 60 → 100점	대화 묶음 수에 더해 약 1만 4천 개라는 트윗 수까지 답했다. 참조 답안에 있던 규모 정보를 보완해 사용자가 확인할 수 있는 정보가 늘었다.
데이터셋에는 어떤 언어가 포함되는가?	`English, German, French` → `English` · 50 → 100점	참조 답안은 영어였다. 독일어·프랑스어를 잘못 덧붙인 부분이 없어져 데이터셋의 언어 범위를 정확하게 전달했다.
트윗은 어떤 언어로 작성됐는가?	`English` → `unanswerable` · 100 → 0점	참조 답안과 맞던 영어라는 답변이 '답할 수 없음'으로 바뀌었다. 사용자가 얻을 수

질문	Qwen2.5에서 Qwen3.8로 바뀐 응답과 F1	사용자에게 어떤 의미가 있는가
		있던 정보를 새 모델이 제공하지 못한 사례다.

앞의 두 사례는 F1 상승이 실제로 **필요한 정보를 더 담거나 잘못 추가한 정보를 줄이는 이득**과 연결된 경우다. 세 번째 사례에서는 모델 교체로 정보 제공이 후퇴했다. 이 사례들은 점수 변화의 의미를 보여주며, 각 유형이 전체에서 차지하는 빈도는 따로 분류하지 않았다.

또한 **답변의 뜻이 사실상 같아도 표기 차이만으로 점수가 오르는 경우**가 있었다. 번역 언어 쌍을 묻는 문항에서 `German–English`가 `German—English`로 바뀌자 F1은 0점에서 100점이 됐다. 이 채점 구현은 두 종류의 대시를 다르게 처리하므로 참조 답안과의 단어 일치가 달라졌다. 사용자가 얻는 독일어·영어라는 정보는 사실상 같아서, 이 상승은 정보가 더 충실해진 사례와 구분해야 한다. **평균 +2.32점에는 실제 정보 개선과 표현 일치의 효과가 함께 들어 있다.** 질문·참조 답안, Qwen2.5 응답, Qwen3.8 문항별 응답

평균 F1 차이 **+2.32점**의 논문 단위 짝지은 부트스트랩 95% 구간은 **+0.57 ~ +4.17점**이다. 273개 논문 묶음을 10,000번 재표집했으며, 교체 실험의 집계 코드와 같은 seed 1을 사용해 다시 확인했다. 이는 평가에 포함되는 논문의 구성을 바꿔 다시 계산했을 때, 평균 차이의 가운데 95% 범위가 모두 새 모델에 유리했다는 뜻이다. **평균 점수 상승이 특정 논문 몇 개의 구성에 얼마나 민감한지 확인하는 근거**로 읽을 수 있다. 개별 질문의 개선 여부는 위 문항별 결과로 봐야 하며, 이 구간이 다루는 불확실성의 범위는 현재 자료의 재표집에 한정된다.

참고로 같은 모델 교체 기록에는 별도의 R0 코드 실험도 있다. 소형 코드는 96문항 중 정답이 **26 → 62개**, 대형 코드는 **25 → 61개**로 늘었다. 각 조건은 세 번 반복했으며, 누적 결과로는 각각 **78/288 → 186/288**, **75/288 → 183/288**이다. 두 조건 모두 정답률 **+37.5%p**지만, 이는 논문 질의응답의 F1과 다른 지표·다른 문제 집합이므로 합산하지 않는다. 포인터 조건과 512토큰 조건을 포함한 별도 상세 결과는 모델 교체 비교 보고서에 있다.

4.4 이번 결과로 판단할 수 있는 범위

입력 절약을 원한다면 R0의 효과가 확인됐고, 같은 검색 원문에서 답변 품질을 조금 더 높이는 데에는 Qwen3.8 구성의 이득이 관측됐다. 다만 전자는 원문 전체보다 품질이 낮아지는 절충이 있었고, 후자는 평균 답변 호출 시간이 약 3.44배로 늘어나는 비용이 있었다. 더 높은 평균 F1과 더 빠른 응답 중 어느 쪽이 중요한지에 따라 선택이 달라진다.

이번 교체 실험에서는 **Qwen3.8의 A·B·FULL을 새로 평가하지 않았다.** 따라서 Qwen3.8에서도 R0가 다른 방식보다 유리하다거나, 같은 모델의 원문 전체 입력 대비 절감률이 77.2%라고 말할 수 없다. 77.2%는 3절의 Qwen2.5 조건에서 측정한 값이다. 또한 R0가 필요한 근거를 빠뜨린 비율은 이번 QASPER 비교에서 별도로 측정하지 않았으므로, 남은 오류의 원인을 전부 검색 또는 모델 중 하나로 돌릴 수 없다.

4.5 결과를 확인할 수 있는 기록

확인할 내용	근거
7종의 판본과 선정 이유	기존 데이터셋 조사, QASPER 선정 기록
Qwen2.5 네 방식의 수치·설정·검증	최종 집계, 실험 설정, 전수 검증
Qwen3.8의 실제 응답과 실행 구성	원자료 ZIP 안의 `results/identity.json`, `results/qasper/qasper-r0-cases/`, `run/osc-q38-qasper.py`, `run/osc-q38-run.py`

확인할 내용	근거
이번 보고서 작성 시 다시 확인한 내용	984문항 대조·재채점 기록: 질문 집합, 원문·지시문 확인값, 신·구 F1, 입력·시간 기록, 95% 구간

이번 재확인은 **저장된 기록과 계산이 서로 맞는지 확인한 것**이다. Qwen3.8을 재실행하거나 새 모델의 토크나이저로 입력을 다시 센 검증은 아니며, 모델 자체의 성능을 외부 기관이 인증한 것도 아니다. 기존 실험 자료와 상세 보고서는 보존하고, 이 문서에서 용어·비교 조건·결과의 의미를 하나의 흐름으로 정리했다.